Functional Requirement Document / Technical Specification Document

Project Name: **InsureDataPipelineData**: October, 2025

Revision Sheet

Release No.	Date	Revision Description	Owner
1	13-Oct-2025	First Version of InsureDataPipelineDataData Intelligence: Sept, 2025	IntelliBI

1.0 Purpose

The purpose of this document is to define the functional and technical requirements for building a **InsureDataPipeline (IDP)** using **Python, Pandas**, **File Automations** aimed at Insurance data from multiple source files into a centralized persistent system.
Using Python file processing and automation will be handled.

The primary objectives include:

- Automate file flow, Validation and archival of files.
- Applying **business logic** to maintain **data quality** and consistency across various datasets.
- Enabling **real-time Analytics and** through Python Script.

2.0 Scope

The scope of this project includes:

- **File Validation** from source systems like flat files (.csv)
- **Data Cleaning** and load it into a preprocessed Layer.
- **Load the data** into a semantic layer for tracking aggregate data.

3.0 Assumptions and Constraints

Assumptions:

- **Business Rules**: Clear business rules for data validation are defined.
- **Data Quality**: Source systems provide accurate data with minimal errors.
- **Tool Availability**: (Python, Pandas etc.) are pre-configured.
- **Stakeholder Engagement**: Business stakeholders are available for input.

Constraints:

1. **Data Volume**: High volumes of data may strain Python resources.

2. **Real-time Reporting**: Near real-time, not true real-time reporting due to processing delays.
3. **Data Source Delays**: Pipeline depends on timely data from external sources.

4.0 DESIGN CONSIDERATIONS

4.1 Naming Standards

FileName:

Convention: `lower_case` (all lowercase, words separated by underscores)

Example : data_ingestion.py

FunctionName:

Convention: `lower_case` (all lowercase, words separated by underscores)

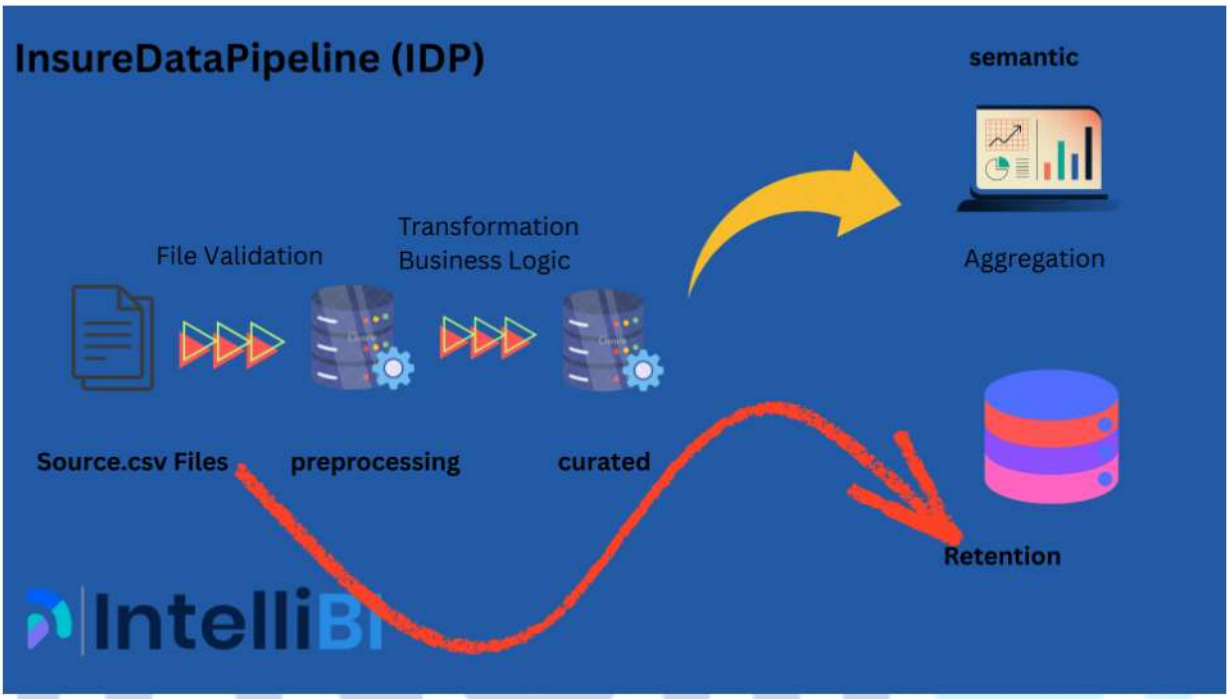
Example : def transform_claim_data():

FolderName:

Convention: `lower_case` (all lowercase, words separated by underscores)

Example : **semantic_layer**

5.0 HIGH Level Design (project architecture DIAGRAM)



Technology components

Technology	Function
Python	For File Validation, Archival
Ad.Python	Build PipeLine and Process the data.

Environments Setup: Development

Create Folder Structure as follows for **dev** Environment:

```
devInsureDataPipeline/
├── config/
│   └── control_files/            — Control files
│       ├── claims_yyyymmdd.json
│       ├── customers_yyyymmdd.json
│       ├── policies_yyyymmdd.json
│       ├── agents_yyyymmdd.json
│       └── payments_yyyymmdd.json
├── data/
│   ├── ingestion/                — Raw incoming files
│   │   ├── claims_yyyymmdd.csv
│   │   ├── customers_yyyymmdd.csv
│   │   ├── policies_yyyymmdd.csv
│   │   ├── agents_yyyymmdd.csv
│   │   └── payments_yyyymmdd.csv
│   ├── preprocessed/             — Cleaned intermediate data
│   │   ├── claims_yyyymmdd.parquet
│   │   ├── customers_yyyymmdd.parquet
│   │   ├── policies_yyyymmdd.parquet
│   │   ├── agents_yyyymmdd.parquet
│   │   └── payments_yyyymmdd.parquet
│   ├── curated/                  — Final merged outputs
│   │   └── claims_enriched.parquet
│   ├── semantic/                 — Business metrics / reporting data
│   │   ├── Policies_Total_ClaimAmt.parquet
│   │   └── Cities_Total_ClaimAmt.parquet
│   └── retention/
│       └── archive/archive_yyyymmdd.zip — Archived old data
├── logs/
│   ├── ingestion_log_yyyymmdd.csv      — SQLite-based audit or log database
│   ├── preprocess_log_yyyymmdd.csv     — Text/CSV audit logs
│   └── retention_log_yyyymmdd.csv      — Error tracebacks or pipeline issues
└── src/
    ├── main.py                 — Entry point orchestrator
    ├── ingestion_manager.py    — File validation and ingestion logic
    ├── preprocessing_engine.py — Preprocessing / cleaning logic
    ├── transformation_engine.py — Transformation and business rule application
    ├── audit_logger.py         — Logging and audit utilities
    ├── retention_manager.py    — Archival and retention policy
    └── utils.py                — Common helper functions
```

Create Folder Structure as follows for **QA** Environment:

1.

Create Folder Structure as follows for **Prod** Environment:

2.

Note: Set up only in the **DEV** environment for now.

1. Inbound Layer:

1. Inbound Layer Requirements

1. **File Validation Control Files**

   - **devInsureDataPipeline/config/control_files/**
     ├── claims_yyyymmdd.json
     ├── customers_yyyymmdd.json
     ├── policies_yyyymmdd.json
     ├── agents_yyyymmdd.json
     ├── payments_yyyymmdd.json

2. **File Validation Data Files**

   - ├── claims_yyyymmdd.csv
     ├── customers_yyyymmdd.csv
     ├── policies_yyyymmdd.csv
     ├── agents_yyyymmdd.csv
     ├── payments_yyyymmdd.csv

3. **Files to Fetch from Source**:

   Source Files Location: [Source Files]

   Control Files Location: [Control Files]

Files to be Validated based on Control file and Data file:

- If File is valid then load the data into the preprocessed layer.

4. **Preprocessed Layer Data Load**:

   Flat File :
   Load the data from the **Source Files** to the **Preprocessed** layer with some basic data cleaning and business logic and audit Columns.

   1. Remove duplicate values .
   2. Add **ingestion_date** column and load system date into **load_date** column.
   3. Add **file_date** column and load file date into it.

   Load Type: Truncate & Load
   Unit Test Case Documents: [Unit Test Case Document Template]
   Data Model: [Preprocessed Layer Data Model]
   STTM :[STTM Source to Preprocessed Layer]

IntelliBI

5. **Preprocessed to Curated Data Load:**

   Load the data from the RAW layer (**Preprocessed**) to the (**Curated)** layer with some basic business logic Columns.

   Load Type: Incremental Load

   STTM Preprocessed to Curated: **STTM_Preprocessed to Curated**
   Data Model: **Cleansed Layer Data Model**

6. **Curated to Semantic Layer Tables:**

   Load Type: Incremental Load
   Please refer to the attached Document for this.

   Unit Test Case Documents: **STTM_Curated to Semantic**
   Data Model: **Refined Layer Data Model**

IntelliBI

Orchestration For Flat File Source:

1. File Validation & Source Files To Preprocessed:

1. **Create end-to-end pipeline (ingestion_manager.py) for each .csv File :**
    - Create function **validate_file()**
    - Validate the files and process only valid files.

2. **Create end-to-end pipeline (preprocessing_engine.py) :**
    - Create function **preprocess_file()**
    - Load the data into parquet files with audit columns.

3. **Create end-to-end pipeline (transformation_engine.py) :**
    - Create function **build_curated_and_semantic()**
    - Load the data into parquet files with required business logic.

4. **Create end-to-end pipeline (retention_manager.py) :**
    - Create function **archive_file()**
    - Move the processed files into the retention/archive folder.

5. **Create end-to-end pipeline (audit_logger.py) :**
    - Create function **archilog_ingestion(), log_preprocess(), log_retention().**
    - Generate the log Files
      Log Files Format:

6. **Create end-to-end Master pipeline (main.py) :**
    - Execute the below functions from **run_pipeline()**
    - **run_pipeline(),**
        - **validated_files()**
        - **preprocess_file()**
        - **build_curated_and_semantic()**
        - **archive_file()**

4. Scheduling:

- Schedule the master pipeline (**main.py**) to run daily at 10 PM IST.

IntelliBI